

T10.1 - Password Hashing

Password Hashing

Contraseñas → información ultra sensible porque pueden permitir identificarse en más de un sistema.

- Se reciben de parte del usuario *cuando crea su cuenta* pero también *para cada autenticación*.
- Se debe poder validar en los registros cada vez que se la necesite.
- Ningún tercero debe poder conseguirlas mediante un ataque.
- Ningún usuario interno con privilegios debería conseguirlas si quisiese.

Solución: *oscurecerlas mediante hashing*.

- Función *determinística* → un texto plano devuelve siempre el mismo hash.
- Función *no reversible* → a partir de un hash hay múltiples posibles textos planos.
- Es *computacionalmente inviable obtener el texto plano a partir de un hash*.

Password hashing permite:

- *Calcular y almacenar el hash al momento de crear la contraseña.*
- *Calcular el hash en cada autenticación y validarlo* con respecto a la versión almacenada.
- *Garantizar la privacidad de una contraseña* ante una filtración de información o el acceso de un superusuario.

Rainbow Tables

Este método no es inmune a ataques de **rainbow tables**, que son *listas de hashes precalculados que sirven para descifrar contraseñas comunes*.

Estas comparan muchos hashes muy rápido para encontrar matches.

No pueden descifrar todas las contraseñas, pero sí una gran parte.

Salting

Es una *técnica para dificultar el ataque por rainbow tables*.

Qué hace:

1. Para cada contraseña a almacenar, se genera un string al azar. Comúnmente llamado salt.

2. Se almacena el salt junto con la información de la cuenta.
3. Cada vez que se tenga que calcular el hash (al crear o validar la contraseña), se concatena el salt con la contraseña original.

Username	Contraseña	Salt	Contraseña_md5	Contraseña a hashear
juanielosegui	contraseñaElosegui	CqiKKuh	d0cd15cf89946637	contraseñaEloseguiCo

Notar que el hash se aplica sobre la contraseña a hashear (contraseña + salt).